

HyperKAN: Hypergraph Representation Learning with Kolmogorov-Arnold Networks

Xiangfei Fang^{1,2}, Boying Wang^{3*}, Chengying Huan⁴, Shaonan Ma⁵, Heng Zhang¹, Chen Zhao¹

¹Institute of Software, Chinese Academy of Sciences, ²University of Chinese Academy of Sciences,

³North University of China, ⁴Nanjing University, ⁵Qiyuan Lab

Abstract—Hypergraph representation learning has garnered increasing attention across various domains due to its capability to model high-order relationships. Traditional methods often rely on hypergraph neural networks (HNNs) employing message-passing mechanisms to aggregate vertex and hyperedge features. However, these methods are constrained by their dependence on hypergraph topology, leading to the challenge of imbalanced information aggregation, where high-degree vertices tend to aggregate redundant features, while low-degree vertices often struggle to capture sufficient structural features.

To overcome the above challenges, we introduce HyperKAN, a novel framework for hypergraph representation learning that transcends the limitations of message-passing techniques. HyperKAN begins by encoding features for each vertex and then leverages Kolmogorov-Arnold Networks (KANs) to capture complex nonlinear relationships. By adjusting structural features based on similarity, our approach generates refined vertex representations that effectively addresses the challenge of imbalanced information aggregation. Experiments conducted on the real-world datasets demonstrate that HyperKAN significantly outperforms state-of-the-art HNN methods, achieving nearly a 9% performance improvement on the Senate dataset.

Index Terms—Kolmogorov-Arnold Networks, Hypergraph Representation Learning.

I. INTRODUCTION

A hypergraph, as a generalized form of a graph where edges (hyperedges) can connect more than two vertices, excels in representing complex relationships and is widely applied in real-world scenarios such as social networks [1]–[3], biological systems [4]–[6], and computer vision [7]–[9], where data often exhibit multi-way relations that need to be effectively modeled. To better capture these higher-order relationships and improve the performance of machine learning models on complex data, numerous methods for hypergraph representation learning have been extensively explored [10]–[12].

Most of these methods [13]–[15] focus on enhancing hypergraph representation learning by designing novel hypergraph neural network (HNN) architectures. These approaches employ operators such as convolution, attention, and diffusion models to facilitate more efficient vertex feature learning. A prevalent strategy is to refine the two-stage message-passing mechanism, which involves first aggregating features from

vertices to hyperedges and then updating features from hyperedges back to vertices. Through this iterative message-passing mechanism, vertices in the hypergraph can gather essential information from their neighbors and the surrounding structure, thereby optimizing representation learning for various tasks.

However, the message-passing mechanism is fundamentally constrained by its reliance on the hypergraph’s original topology, leading to the challenge of imbalanced information aggregation. This issue arises from the degree heterogeneity in hypergraphs and the mechanism’s limited focus on vertex-to-hyperedge and hyperedge-to-vertex interactions. As a result, vertices with few neighbors may receive insufficient information, while vertices with many neighbors can be overloaded with redundant information. This imbalance in information flow adversely affects the learning process, ultimately resulting in suboptimal performance in downstream tasks.

To address this challenge, we propose a novel framework for hypergraph representation learning, termed HyperKAN. Unlike traditional methods based on message-passing mechanisms, HyperKAN initially extracts feature encodings for each vertex from the hypergraph and then utilizes these encoded features as inputs to Kolmogorov-Arnold Networks (KANs) for effective vertex representation learning. We start by computing each vertex’s feature vector as a weighted sum of its initial features and those of its neighbors. To tackle the issue of imbalanced information aggregation, we first adjust the structural features using the similarity features which can reduce redundant features for high-degree vertices and enrich the information for low-degree vertices. We then leverage the powerful dynamic adjustment and adaptability capabilities of KANs to effectively capture complex, nonlinear relationships and further extract higher-order features from these adjusted structural features. This approach further reduces redundancy for high-degree vertices and enhances information acquisition for low-degree vertices, further addressing the challenge of imbalanced information aggregation.

We conduct experiments on four real-world datasets and compare its performance with state-of-the-art HNNs to validate the effectiveness of HyperKAN. The experimental results show that HyperKAN significantly outperforms existing methods, achieving nearly a 9% improvement on the Senate dataset.

II. BACKGROUND AND RELATED WORK

Hypergraph. A hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ consists of a set of vertices $\mathcal{V}$ and hyperedges $\mathcal{E}$, where each hyperedge

*Corresponding Author.

This work was supported in part by National Natural Science Foundation of China (No. 62002350), in part by Youth Innovation Promotion Association CAS (No. 2023120), and in part by the Youth Project of Applied Basic Research Project of Shanxi Province (entitled “Research on General Object Detection Algorithms Driven by Multi-Source Imbalanced Data”).

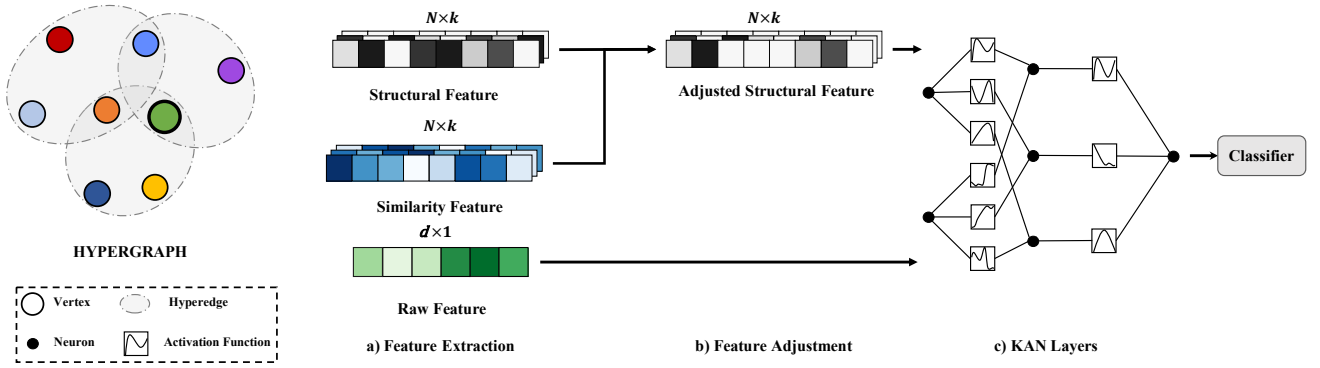


Fig. 1: Architecture of the HyperKAN model where N is the number of vertices, k is the hop number, and d is the dimension of the vertex feature vector. The model comprises three key components: a) the Feature Extraction module, which extracts structural features from the hypergraph structure and similarity features from the raw vertex features; b) the Feature Adjustment module for structural feature adjustment utilizing the similarity features; and c) the KAN layers for further encoding and feature extraction.

$e \in \mathcal{E}$ is a subset of $\mathcal{V}$ and can connect multiple vertices. This structure allows hypergraphs to capture complex, higher-order relationships. The incidence matrix $\mathbf{H}$, where $\mathbf{H}_{ij} = 1$ if vertex i is in hyperedge j and $\mathbf{H}_{ij} = 0$ otherwise, is used to represent these connections efficiently.

Hypergraph Neural Networks. Recent research has focused on developing hypergraph neural networks (HNNs) to efficiently learn representations from hypergraphs. Some approaches convert hypergraphs into standard graphs and apply message-passing techniques for convolution operations on these converted structures [16]–[19]. Other studies have proposed new HNN architectures based on a two-stage message-passing mechanism, such as HGNN [20], UniGNN [13], AllSet [14], and others [21]–[25]. Despite their innovations, these methods are inherently limited by the hypergraph or the converted graph structure, as they typically only capture information from immediate neighbors. This restricts the potential of hypergraph representation learning to fully utilize the rich, higher-order relationships present in the data.

Kolmogorov-Arnold Networks. Kolmogorov-Arnold Networks (KANs) offer a novel approach to representation learning, particularly suited for modeling complex, nonlinear relationships. Unlike traditional neural networks like Multi-Layer Perceptrons (MLPs) that use fixed activation functions, KANs use learnable activation functions parameterized as splines, allowing them to adapt to diverse data patterns and enhance modeling flexibility and accuracy [26]. Recently, KANs have been applied to graph-structured data [27]–[29], but their use for hypergraph-structured data remains unexplored. To the best of our knowledge, our work is the first to employ KANs for hypergraph representation learning.

III. PROPOSED HYPERKAN ARCHITECTURE

We introduce a novel architecture, named **HyperKAN**, for hypergraph representation learning, as illustrated in Fig. 1. HyperKAN starts by leveraging the connectivity and similarity properties of vertices in the hypergraph to extract two key types of features: structural features and similarity features

(Section III-A). To address the challenge of imbalanced information aggregation, HyperKAN applies an adjusting process on the structural features using similarity (Section III-B). The adjusted structural features are then encoded and processed through Kolmogorov-Arnold Network (KAN) layers to capture deeper, more complex relationships (Section III-C). Finally, the vertex classification layer generates the final classification result (Section III-D).

A. Hypergraph Feature Extraction Module

Structural feature. A vertex $v_i \in \mathcal{V}$ gathers features from its neighboring vertices connected by hyperedges, which can be represented using the adjacency matrix of vertices, $A = \mathbf{H}\mathbf{H}^\top$. Building on this adjacency matrix, we extract k -hop neighbor features to leverage the hypergraph’s structural properties, resulting in a comprehensive structural feature representation formulated as $h_i = [A_i, A_i^2, A_i^3, \dots, A_i^k]$, where A_i represents the i^{th} row of the adjacency matrix A , which corresponds to the direct neighbors of vertex v_i , A_i^2 refers to the i^{th} row of A^2 indicating the 2-hop neighbors of v_i , and this pattern continues up to A_i^k for the k -hop neighbors.

The structural feature vector h_i encapsulates multi-hop neighborhood information for vertex v_i , providing a richer representation of the hypergraph’s topology. Unlike the vertex’s original features, these features derived from its neighbors encapsulate the structural context of the hypergraph, offering a more comprehensive understanding of relationships within it. This enriched representation enhances the model’s ability to capture complex dependencies and interactions, which is crucial for subsequent classification tasks.

Similarity feature. To further enhance the representation of vertices in the hypergraph, we compute similarity features by evaluating how each vertex relates to all others. For this, we calculate a similarity score between each pair of vertices to form a graph similarity matrix $S \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, where each element $S(i, j)$ indicates the similarity score between vertices v_i and v_j . To efficiently construct this matrix, we use cosine similarity. Let $X \in \mathbb{R}^{n \times d}$ be the feature matrix, where each row $X_i \in \mathbb{R}^d$ represents the feature vector of vertex v_i in a

d -dimensional space. The cosine similarity matrix $S \in \mathbb{R}^{n \times n}$ is defined to measure the similarity between pairs of vertices based on their feature vectors in X . Specifically, each entry S_{ij} in matrix S is computed as:

$$S_{ij} = \frac{X_i \cdot X_j}{\|X_i\| \|X_j\|}, \quad (1)$$

where X_i and X_j are the feature vectors of vertices v_i and v_j , respectively; $\cdot$ denotes the dot product, and $\|\cdot\|$ denotes the Euclidean norm. This cosine similarity matrix S captures the pairwise similarities between vertices based on the angles between their feature vectors. The similarity features of a vertex v_i are given by the row vector s_i of matrix S , where $s_i \in \mathbb{R}^n$ contains the cosine similarity values between vertex v_i and all other vertices. This representation effectively reflects vertex relationships using their raw feature information.

B. Feature Adjustment Module

To address the challenge of imbalanced information aggregation, we employ an adjustment mechanism based on similarity features to refine the structural features. We design a filtering function $\mathcal{F}$ to adjust the structural feature h_i , reducing redundant neighbors for densely connected vertices while incorporating important multi-hop neighbors for sparsely connected ones. We use the feature adjustment of A_i as an illustrative example.

If the j^{th} element of A_i is non-zero, it indicates a connection between vertex v_i and vertex v_j . To mitigate redundancy from multiple neighboring vertices, we utilize the similarity features S_i for adjustment. Specifically, if a vertex has more than n neighbors, we retain only the top- n neighbors with the highest similarity scores and remove those with lower similarity. For such cases, we set $A_{ij} = 0$ if $S_{ij} \notin \text{top-}n(S_i)$. After adjusting, a vertex will retain features from at most n neighbors that are most similar to its own features.

Conversely, if a vertex has fewer than m direct neighbors, we add extra neighbors to its structural feature based on the similarity features S_i . We select additional neighbors that have the most similar characteristics based on the similarity scores. For each vertex v_i , its similarity feature S_i is used for adjustment as follows:

$$\text{Mask}_{ij} = \begin{cases} 1, & \text{if } A_{ij} = 0 \\ 0, & \text{otherwise} \end{cases}, \quad (2)$$

We then set $A_{ij} = 1$ if $S_{ij} \in \text{Top-}m(S_i \cdot \text{Mask}_i)$ where $\cdot$ represents the dot product.

The above processes eliminate redundant structural features for high-degree vertices and add necessary hypergraph structural information for low-degree vertices. Such a balanced and informative set of neighbors is crucial for capturing the underlying hypergraph structure effectively. To formalize these operations, we denote these two aforementioned filtering functions as $\mathcal{F}_1$ and $\mathcal{F}_2$ separately. With these functions in place, the adjusted structural features can be represented as:

$$\hat{A}_i = \frac{\hat{A}_i}{d_i}, \quad (3)$$

where $\hat{A}_i = \mathcal{F}_1(\mathcal{F}_2(A_i))$ and d_i is defined as $d_i = \sum_j \hat{A}_{ij}$. Similarly, the remaining components of h_i are adjusted, resulting in a refined structural feature denoted as $\hat{h}_i = [\hat{A}_i, \hat{A}_i^2, \hat{A}_i^3, \dots, \hat{A}_i^k]$.

C. KAN Layers

Building upon the adjusted structural features defined above, we perform an aggregation with the raw features x of each vertex to enrich the representation by combining both structural and attribute information. The aggregation process for a vertex v_i is defined as:

$$e_i = \text{Aggregate}(\sum_{j=1}^k (\hat{h}_{ij} x_i), x_i). \quad (4)$$

In this paper, we define the aggregation function as the weighted sum of vectors. Subsequently, we map the aggregated vector to a d -dimensional feature space, represented as $z_i = \text{Embed}(e_i) \in \mathbb{R}^{d \times 1}$. The features of all vertices are organized into a matrix form $Z^0 = [z_1, z_2, \dots, z_k] \in \mathbb{R}^{k \times d}$, where k is the count of vertices. The matrix Z^0 is then fed into the KAN layers as input to update the vertex representations across multiple layers. After l KAN layers, the initial feature Z^0 is transformed into a higher-level representation Z^l . This process can be mathematically described as:

$$Z^l = (\Phi_{L-1} \circ \dots \circ \Phi_1 \circ \Phi_0) Z^0, \quad (5)$$

where Φ_L represents a KAN layer and is given by:

$$\Phi_L = \begin{pmatrix} \phi_{l,1,1}(\cdot) & \phi_{l,1,2}(\cdot) & \cdots & \phi_{l,1,n_l}(\cdot) \\ \phi_{l,2,1}(\cdot) & \phi_{l,2,2}(\cdot) & \cdots & \phi_{l,2,n_l}(\cdot) \\ \vdots & \vdots & \ddots & \vdots \\ \phi_{l,n_l+1,1}(\cdot) & \phi_{l,n_l+1,2}(\cdot) & \cdots & \phi_{l,n_l+1,n_l}(\cdot) \end{pmatrix}, \quad (6)$$

where each $\phi_{l,i,j}$ is the activation function that connects i^{th} neuron in the l^{th} layer and the j^{th} neuron in the $l+1^{th}$ layer.

D. Loss Function

For the vertex classification task, we train our model using the cross-entropy loss function, which is defined as:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C \mathbf{y}_{i,c} \log(\hat{\mathbf{y}}_{i,c}), \quad (7)$$

where N is the total number of labeled vertices, C is the number of classes, $\mathbf{y}_{i,c}$ represents the ground-truth label of i -th sampled vertex, and $\hat{\mathbf{y}}_{i,c}$ represents the prediction value.

IV. EVALUATION

A. Experimental settings

Datasets. We utilize four datasets to evaluate our model's performance on the node classification tasks. The first dataset is a hypergraph-based citation network, Citeseer [16]. Another dataset is an adaptation of the Senate data [30], as proposed in [31]. The final two datasets, ModelNet40 [32] and NTU2012

[33], are used for visual object classification tasks. Table I provides a comprehensive overview of these datasets.

Baselines. The performance of the model is evaluated through classification tasks on hypergraph vertices. We compare our approach against several baseline models, including HGNN [20], HyperGCN [16], UniGNN [13], HNHN [15], HCHA [23], AllSet [14], HGNN⁺ [34], and EDHNN [24]. All these baselines have open-source implementations available, ensuring that the results are reproducible.

TABLE I: Dataset statistics in our evaluation.

Dataset	#Vertices	#Edges	#Vector Dimensions	#Classes
Citeseer	3327	9464	500	6
Senate	282	315	100	2
ModelNet40	12311	12311	100	40
NTU2012	2012	2012	100	67

Experiment Settings. We adhere to the standard practice of partitioning each dataset into training, validation, and testing sets, following a ratio of 2 : 1 : 1. For optimization, we use the AdaGrad algorithm [35]. To ensure the robustness and reliability of our results, each model is trained and validated ten times with different configurations. The evaluation metric used is prediction accuracy. All experiments are conducted using the PyTorch framework on an RTX 3090 GPU.

TABLE II: Performance comparison with state-of-the-art HNNs on hypergraph datasets: mean accuracy (%) $\pm$ standard deviation. **Bold font** indicates the best result, while best baselines are underlined.

Methods	Citeseer	Senate	ModelNet40	NTU2012
HGNN	73.07% $\pm$ 1.35	59.72% $\pm$ 4.61	95.27% $\pm$ 0.37	86.74% $\pm$ 1.31
HyperGCN	71.52% $\pm$ 1.28	52.68% $\pm$ 7.72	75.07% $\pm$ 3.26	53.76% $\pm$ 4.63
HNHN	72.13% $\pm$ 1.43	53.52% $\pm$ 6.04	97.19% $\pm$ 0.35	88.65% $\pm$ 1.30
HCHA	72.48% $\pm$ 1.36	51.63% $\pm$ 4.16	94.60% $\pm$ 0.29	87.41% $\pm$ 1.64
UniGNNII	73.13% $\pm$ 1.30	54.23% $\pm$ 5.68	97.50% $\pm$ 0.31	88.77% $\pm$ 0.89
HGNN ⁺	73.02% $\pm$ 1.43	55.63% $\pm$ 6.88	94.35% $\pm$ 0.39	86.66% $\pm$ 1.33
AllSetTransformer	73.08% $\pm$ 0.74	53.81% $\pm$ 5.30	<u>98.11% $\pm$ 0.26</u>	88.67% $\pm$ 1.29
AllDeepSets	70.08% $\pm$ 1.72	56.90% $\pm$ 5.22	96.81% $\pm$ 0.36	88.39% $\pm$ 1.25
EDHNN	<u>74.03% $\pm$ 1.27</u>	<u>65.49% $\pm$ 2.02</u>	97.28% $\pm$ 0.27	87.95% $\pm$ 0.94
HyperKAN	75.05% $\pm$ 1.16	76.37% $\pm$ 6.84	98.52% $\pm$ 0.18	90.58% $\pm$ 1.48

B. Experiment Results

Comparison with SOTA methods. Table I presents a comparative analysis of our HyperKAN model against ten hypergraph baselines. The results indicate that, while the baseline models show varying strengths and weaknesses across different datasets, HyperKAN consistently achieves superior performance, outperforming all state-of-the-art models. Notably, on the Senate dataset, HyperKAN surpasses the second-best model by nearly 9% on average. This substantial gain in accuracy is primarily attributed to two factors. This significant improvement in accuracy is primarily due to two factors. First, our feature extraction and adjustment mechanism effectively balances the feature acquisition capabilities between high-degree and low-degree vertices. This approach allows low-degree vertices to capture more meaningful features while preventing high-degree vertices from accumulating redundant information. Second, the Kolmogorov-Arnold Networks (KANs) enhance the model’s capacity to capture complex, nonlinear

relationships, providing a deeper understanding of high-order associations within hypergraphs. The additional encoding by KAN layers allows our model to better interpret these intricate relationships, leading to higher overall accuracy.

TABLE III: Result of the ablation study.

Dataset	w/o both	w/o FE&FA	w/o KAN	with both
Citeseer	71.22% $\pm$ 1.16	73.50% $\pm$ 1.61	73.65% $\pm$ 0.93	75.05% $\pm$ 1.16
NTU2012	87.80% $\pm$ 2.05	88.89% $\pm$ 1.49	89.86% $\pm$ 1.91	90.58% $\pm$ 1.48

Ablation Study. We conduct a series of ablation studies to assess the importance of various modules in our HyperKAN model using two selected datasets. We create two model variants: HyperKAN (w/o FE&FA), which excludes the Feature Extraction (FE) and Feature Adjustment (FA) modules, and HyperKAN (w/o KAN), which omits the KAN layers. Additionally, we consider HyperKAN (w/o both), which removes all the modules. The results of these ablation studies, presented in Table III, demonstrate that both HyperKAN (w/o FE&FA) and HyperKAN (w/o KAN) perform worse than the full HyperKAN model, underscoring the essential role of the FE, FA, and KAN modules. Notably, HyperKAN (w/o both) shows the poorest performance, highlighting the critical importance of our complete approach in enhancing model discriminability.

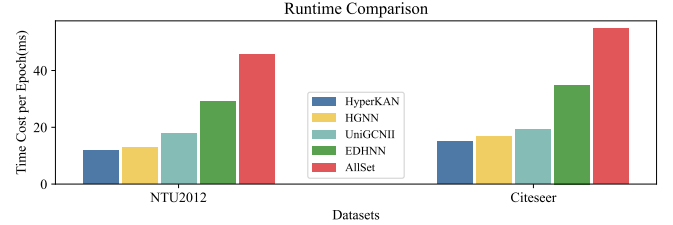


Fig. 2: Comparison of runtime.

Runtime Comparison. We conducted experiments on the NTU2012 and Citeseer datasets to compare the runtime speed of popular HNN models with HyperKAN, as shown in Fig. 2. The results show that HyperKAN’s runtime is comparable to that of HGNN, yet it significantly outperforms HGNN in accuracy. Furthermore, HyperKAN demonstrates superior performance compared to state-of-the-art methods, including EDHNN, AllSet, and UniGNNII, while also maintaining lower computational costs. This highlights HyperKAN’s efficiency in achieving high accuracy without significantly increasing the total resource consumption.

V. CONCLUSION

In this work, we introduce HyperKAN, a novel framework for hypergraph representation learning. Our approach begins by extracting essential structural features from the hypergraph, followed by employing Kolmogorov-Arnold Networks (KANs) for deeper encoding and feature extraction. This method enables high-degree vertices to avoid redundant features, while low-degree vertices can effectively acquire sufficient structural information. Extensive experiments on real-world datasets demonstrate that HyperKAN consistently outperforms state-of-the-art methods, underscoring its effectiveness and efficiency in hypergraph representation learning.

REFERENCES

- [1] D. Li, Z. Xu, S. Li, and X. Sun, "Link prediction in social networks based on hypergraph," in *Proceedings of the 22nd international conference on world wide web*, 2013, pp. 41–42.
- [2] C. Meng and H. Motevalli, "Link prediction in social networks using hyper-motif representation on hypergraph," *Multimedia Systems*, vol. 30, no. 3, p. 123, 2024.
- [3] Y. Guan, X. Sun, and Y. Sun, "Sparse relation prediction based on hypergraph neural networks in online social networks," *World Wide Web*, vol. 26, no. 1, pp. 7–31, 2023.
- [4] S. Feng, E. Heath, B. Jefferson, C. Joslyn, H. Kvinge, H. D. Mitchell, B. Praggastis, A. J. Eisfeld, A. C. Sims, L. B. Thackray *et al.*, "Hypergraph models of biological networks to identify genes critical to pathogenic viral response," *BMC bioinformatics*, vol. 22, no. 1, p. 287, 2021.
- [5] S. Jin, Y. Hong, L. Zeng, Y. Jiang, Y. Lin, L. Wei, Z. Yu, X. Zeng, and X. Liu, "A general hypergraph learning algorithm for drug multi-task predictions in micro-to-macro biomedical networks," *PLOS Computational Biology*, vol. 19, no. 11, p. e1011597, 2023.
- [6] J. Lugo-Martinez, D. Zeiberg, T. Gaudelet, N. Malod-Dognin, N. Przulj, and P. Radivojac, "Classification in biological networks with hypergraphlet kernels," *Bioinformatics*, vol. 37, no. 7, pp. 1000–1007, 2021.
- [7] W. Shi and R. Rajkumar, "Point-gnn: Graph neural network for 3d object detection in a point cloud," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 1711–1719.
- [8] P. Pradhyumna, G. Shreya *et al.*, "Graph neural network (gnn) in image and video understanding using deep learning for computer vision applications," in *2021 Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021, pp. 1183–1189.
- [9] K. Han, Y. Wang, J. Guo, Y. Tang, and E. Wu, "Vision gnn: An image is worth graph of nodes," in *Advances in Neural Information Processing Systems*, 2022.
- [10] A. Antelmi, G. Cordasco, M. Polato, V. Scarano, C. Spagnuolo, and D. Yang, "A survey on hypergraph representation learning," *ACM Computing Surveys*, vol. 56, no. 1, pp. 1–38, 2023.
- [11] Z. Liu, B. Tang, Z. Ye, X. Dong, S. Chen, and Y. Wang, "Hypergraph transformer for semi-supervised classification," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 7515–7519.
- [12] S. Kim, S. Kang, F. Bu, S. Y. Lee, J. Yoo, and K. Shin, "Hypeboy: Generative self-supervised representation learning on hypergraphs," *arXiv preprint arXiv:2404.00638*, 2024.
- [13] J. Huang and J. Yang, "Unignn: a unified framework for graph and hypergraph neural networks," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, 2021.
- [14] E. Chien, C. Pan, J. Peng, and O. Milenkovic, "You are allset: A multiset function framework for hypergraph neural networks," in *International Conference on Learning Representations*, 2022.
- [15] Y. Dong, W. Sawin, and Y. Bengio, "Hnhn: Hypergraph networks with hyperedge neurons," *arXiv preprint arXiv:2006.12278*, 2020.
- [16] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "Hypergcnn: A new method for training graph convolutional networks on hypergraphs," *Advances in neural information processing systems*, vol. 32, 2019.
- [17] L. Sun, S. Ji, and J. Ye, "Hypergraph spectral learning for multi-label classification," in *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2008, pp. 668–676.
- [18] C. Yang, R. Wang, S. Yao, and T. Abdelzaher, "Semi-supervised hypergraph node classification on hypergraph line expansion," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 2352–2361.
- [19] Y. Yan, Y. Chen, S. Wang, H. Wu, and R. Cai, "Hypergraph joint representation learning for hypervertices and hyperedges via cross expansion," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 8, 2024, pp. 9232–9240.
- [20] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, 2019, pp. 3558–3565.
- [21] D. Arya, D. K. Gupta, S. Rudinac, and M. Worring, "Hypersage: Generalizing inductive representation learning on hypergraphs," *arXiv preprint arXiv:2010.04558*, 2020.
- [22] F. Tudisco, K. Prokopcik, and A. R. Benson, "A nonlinear diffusion method for semi-supervised learning on hypergraphs," *arXiv preprint arXiv:2103.14867*, 2021.
- [23] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognition*, vol. 110, p. 107637, 2021.
- [24] P. Wang, S. Yang, Y. Liu, Z. Wang, and P. Li, "Equivariant hypergraph diffusion neural operators," in *International Conference on Learning Representations (ICLR)*, 2023.
- [25] I. Duta, G. Cassarà, F. Silvestri, and P. Liò, "Sheaf hypergraph networks," 2023.
- [26] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.
- [27] Bresson, "Kagenns: Kolmogorov-arnold networks meet graph learning," *arXiv preprint arXiv:2406.18380*, 2024.
- [28] G. De Carlo, A. Mastropietro, and A. Anagnostopoulos, "Kolmogorov-arnold graph neural networks," *arXiv preprint arXiv:2406.18354*, 2024.
- [29] M. Kiamari, M. Kiamari, and B. Krishnamachari, "Gkan: Graph kolmogorov-arnold networks," *arXiv preprint arXiv:2406.06470*, 2024.
- [30] J. H. Fowler, "Legislative cosponsorship networks in the us house and senate," *Social networks*, vol. 28, no. 4, pp. 454–465, 2006.
- [31] P. S. Chodrow, N. Veldt, and A. R. Benson, "Hypergraph clustering: from blockmodels to modularity," *arXiv e-prints*, pp. arXiv–2101, 2021.
- [32] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1912–1920.
- [33] D.-Y. Chen, X.-P. Tian, Y.-T. Shen, and M. Ouhyoung, "On visual similarity based 3d model retrieval," in *Computer graphics forum*, vol. 22, no. 3. Wiley Online Library, 2003, pp. 223–232.
- [34] Y. Gao, Y. Feng, S. Ji, and R. Ji, "Hgnn+: General hypergraph neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3181–3199, 2023.
- [35] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.